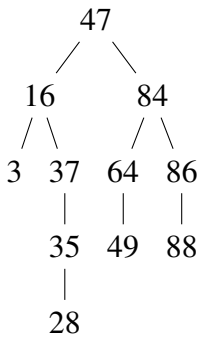
Problem Set 4

Name: Nicolas Iskandar

Collaborators: None

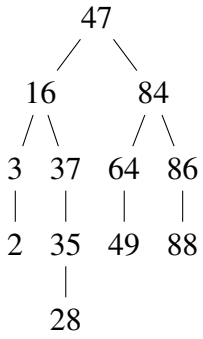
Problem 4-1.

(a) The skew of a node v is $\text{height}(v.\text{right}) - \text{height}(v.\text{left})$; a node is height-balanced iff its skew is in $\{-1, 0, 1\}$. Computing heights bottom-up:

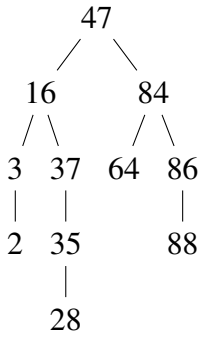


Node 16 has skew $+2$ (right subtree 2 taller) and node 37 has skew -2 (left subtree 2 taller). Every other node is balanced.

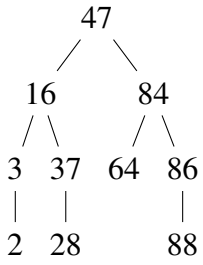
(b) Plain BST insert/delete (no rotations): insert walks down by key comparison to an empty spot; delete of a 2-child node swaps in a predecessor/successor and deletes that (now ≤ 1-child) node instead.



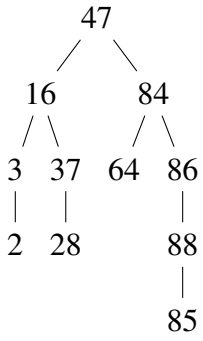
`insert(2)`



`delete(49)` (49 was a leaf)

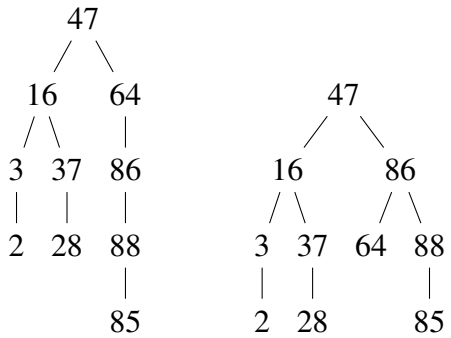


`delete(35)` (one child, splice)



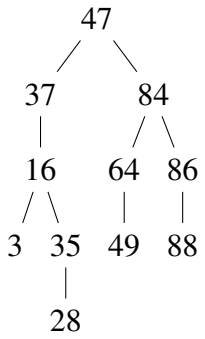
`insert(85)`

`delete(84)`: two children, so swap in either the predecessor (64) or successor (86):

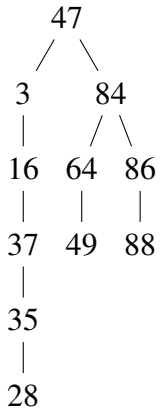


(c) A rotation is an $O(1)$ local restructuring that preserves the BST property. Applied to the original tree from (a):

Node 16 (skew $+2$): both directions fail to balance, since 16's imbalance is a *zig-zag* case (its heavy child is itself skewed the other way), a single rotation can't fix this, only a double rotation could.

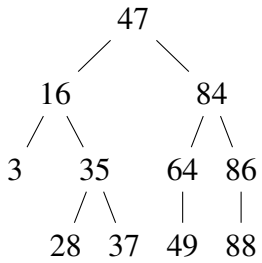


rotate left (still unbalanced)



rotate right (still unbalanced)

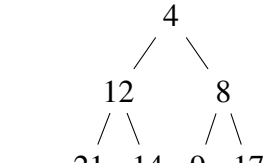
Node 37 (skew -2): a left rotation is impossible (no right child to bring up); a right rotation is a *straight* (left-heavy) case, so it fixes the imbalance:



rotate right at 37: **height-balanced!**

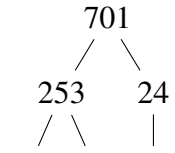
Problem 4-2.

An array $A[0..n-1]$ represents a complete tree with children of $A[i]$ at $A[2i+1], A[2i+2]$; it's a max-heap if every parent $\geq$ its children (min-heap: $\leq$). We fix a non-heap by repeated parent/child swaps.



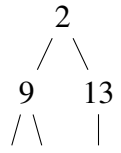
(a)

Every parent $\leq$ its children: **min-heap**.



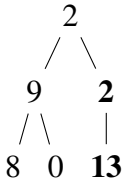
(b)

Every parent $\geq$ its children: **max-heap**.

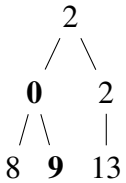


(c)

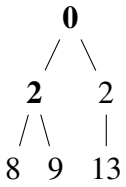
Neither ($9 > 0$ and $13 > 2$ both violate the min-heap property). Bottom-up sift-down fixes it in three swaps:



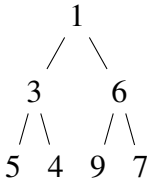
swap $(13, 2)$



swap $(9, 0)$



swap $(2, 0)$ at root: valid **min-heap**



(d)

Every parent $\leq$ its children: **min-heap**.

Problem 4-3.

(a) Build a max-heap on scores from A in $O(|A|)$ time, then call `delete_max()` k times, recording each returned registration number. Correct since `delete_max` always removes a current-maximum item; runs in $O(|A| + k \log |A|)$ since build is $O(|A|)$ and each of the k deletions is $O(\log |A|)$.

(b) Since A is a max-heap, all items with score $> x$ form a connected subtree containing the root. Recursively search from the root: if a node's score is $\leq x$, stop (its whole subtree is $\leq x$); otherwise report it and recurse into both children. This visits only the n_x reported nodes plus at most two children each (which turn out $\leq x$), i.e. $O(n_x)$ nodes total, giving $O(n_x)$ time.

Problem 4-4.

Data structures: a priority queue P of farms keyed on available capacity a_i; a hash table B mapping building name $\mapsto$ (s_i, d_j); a hash table F mapping farm address $\mapsto$ (its own set B_i of connected buildings, a pointer to its location in P). Cross-pointers let every operation avoid searching.

- `initialize`(S): build F, P directly from S in $O(n)$ time; B starts empty.
- `power_on`(b_j, d_j): `delete_max` the farm of largest available capacity from P. If d_j exceeds it, no farm qualifies (reinsert, report failure). Otherwise subtract d_j, reinsert into P (relink pointer in F), and record b_j in B and in that farm's B_i. Cost: $O(\log n)$ (dominated by the heap operations).
- `power_off`(b_j): look up (s_i, d_j) in B, remove b_j from B_i (via F), then use F's pointer to relocate s_i in P, add back d_j capacity, and reinsert. Cost: $O(\log n)$.
- `customers`(s_i): look up B_i via F and return its contents in $O(k)$ time.

Implementation: B, F as hash tables ($O(n)$ build, *expected* $O(1)$ lookup, so `power_on`/`off` are expected/amortized); each B_i as a Set AVL tree or hash table; P as a max-heap or augmented Sequence AVL tree (supports $O(n)$ build, $O(\log n)$ insert/delete-max/delete-by-pointer, deleting an arbitrary heap item by pointer uses the same swap-to-last-leaf technique as deleting the root).

Problem 4-5.

Store the matrices as leaves (in order) of a Sequence AVL tree T; augment each node v with $v.P$, the ordered product of all matrices in its subtree. Since matrix multiplication is associative,

$$v.P = P_L \cdot v.M \cdot P_R,$$

computable in $O(1)$ from children's P (identity matrix for a missing child), so this augmentation is maintained under all normal Sequence AVL operations at no extra asymptotic cost.

- `initialize`(M): build T from M in order in $O(n)$, computing P bottom-up.
- `update_joint`(k, M): `get_at`(k) finds the node in $O(\log n)$; update its matrix, then recompute P for its $O(\log n)$ ancestors.
- `full_transformation`$()$: return $T.\text{root}.P$ in $O(1)$; by construction, it's exactly the full ordered product.

(Since the set/order of matrices never changes, only their values, self-balancing isn't strictly needed; a static or array-based complete tree works too.)

Problem 4-6.

(a) If (x_0, y_0) gives tastiness $t \neq 0$, let x_i be the largest topping x-coordinate $\leq x_0$ and y_j the largest topping y-coordinate $\leq y_0$ (both exist since the slice is nonempty). Since no topping has x- or y-coordinate strictly between these values and x_0, y_0, the slice at (x_i, y_j) contains exactly the same toppings as the slice at (x_0, y_0), hence the same tastiness t. So an optimal slice can always be assumed topping-aligned on both axes.

(b) Augment a Set AVL tree with, at each node v:

- $v.\text{sum} = v.\text{left.sum} + v.\text{item.val} + v.\text{right.sum}$;
- $v.\text{max_prefix}$: the best prefix sum ending in v's subtree. By BST order, a prefix ending in the subtree either ends in the left subtree, ends exactly at v, or ends in the right subtree (which then also includes all of the left subtree and v):

$$v.\text{max_prefix} = \max\left(\begin{array}{l} v.\text{left.max_prefix}, \\ v.\text{left.sum} + v.\text{item.val}, \\ v.\text{left.sum} + v.\text{item.val} + v.\text{right.max_prefix} \end{array}\right);$$

- $v.\text{max_prefix_key}$: the key achieving that max, read off whichever of the three cases won.

All three are $O(1)$ local recomputations, so they're maintained through every Set AVL operation at no extra cost. Then `max_prefix()` just returns

$$(T.\text{root.max_prefix_key}, T.\text{root.max_prefix})$$

in $O(1)$.

(c) Sort toppings by x ($O(n \log n)$). Sweep left to right, inserting each topping's (y_i, t_i) into the part-(b) structure and querying `max_prefix()` after each insertion; this gives, at each x_i, the best achievable tastiness with the x-cut at x_i (since the structure exactly holds the toppings with $x \leq x_i$, keyed by y). Track the best (x_i, y^*, t^*) over the sweep. By part (a), an optimal slice occurs at some such x_i, so this is correct; each of the n steps costs $O(\log n)$ (insert) $+O(1)$ (query), for $O(n \log n)$ total.